

Phase Two Report (80 points)

In your report, provide brief but clear answers to the following questions. Include relevant code snippets and screenshots when appropriate.

1. (15 pts) URL and Tables

a. (5 pts) What is the customers main page url?

Also provide the username and password for one existing customer account. This will help TAs test your system. Provide one existing user username and password.

<https://classdb.it.mtu.edu/~mjdennis/shopMain.php>

Username: adroplett

Password: fatherdroplet

b. (5 pts) What is the employee page url?

Also provide the username and password for one existing employee account. This will help TAs test employee functions. Provide one existing employee username and password.

<https://classdb.it.mtu.edu/~mjdennis/employeeMain.php>

Username: a

Password: a

c. (5 pts) Specify the database name and list all the table names used in your project.

No need to list column names — just include the names of the

The database name is mjdennis

The tables are Category, Customer, ItemInCart, OrderInformation, OrderItem, Product, ProductHistory, Employee

2. (10 pts) Describe how transactions are used in your project.

Include code snippets demonstrating transaction handling (e.g., for checkout or stock updates).

Our transactions were primarily handled by the SQL code we created in part 1 of the project. We need to set the variables to use for checkout. The procedure in SQL needs a customer_id, and will output a order_id, and out_of_stock_product. The customer ID is already handled and the user_id of the current session is used. But a variable for the order_id, and out_of_stock product must be set to null to be used, which is the point of the //Set variables to null section of the code. The checkout procedure from SQL is then called which will go through all the items in the cart and make sure there is enough of the stock, if not it is rolled back and the variable out_of_stock product will be set to the product that is out of stock. The PHP code understands that and gets the missing_item. If it is null it knows the SQL procedure worked correctly it will print that it is a success. If there is a missing_item it understands that it was not a success and will print the missing item and what the amount in stock is. The actual part about changing the Product numbers and emptying the cart, and adding it to OrderInformation are completely handled by the SQL code.

*included full code here because I thought it was needed, used shorter snippets for the others

Unset

```
#Checks to make sure "checkout" button is pressed
if(isset($_POST['checkout'])) {

    // Sets variables to null
    $sql = "set @p_order_id = NULL";
    $pdo->exec($sql);
    $sql = "set @p_out_of_stock_product = NULL";
    $pdo->exec($sql);

    // Uses the code created in part 1
    $sql = "Call checkout(:customer_id, @p_order_id, @p_out_of_stock_product);";
    $stmt = $pdo->prepare($sql);
    $stmt->bindParam(':customer_id', $userid);
    $stmt->execute();

    // Saves the missing item (the only way it would work is as an array)
    $missing_item_array = $pdo->query("Select @p_out_of_stock_product as missing_item;")->fetch(PDO::FETCH_ASSOC);
    $order_id_array = $pdo->query("Select @p_order_id as order_id;")->fetch(PDO::FETCH_ASSOC);

    # Gets the missing item and order id from array
    $missing_item = $missing_item_array['missing_item'];
    $order_id = $order_id_array['order_id'];

    #If there is a missing item it will get the stock quantity of the item
    if($missing_item != null){
        $show_low_array = $pdo->query("Select stock_quantity from Product where product_id = $missing_item;")->fetch(PDO::FETCH_ASSOC);
        $show_low = $show_low_array['stock_quantity'];
    }

    #Prints the missing item and how low the stock is for it
    if($missing_item != null){
        echo "<p> Product ID $missing_item does not have enough in stock (Stock amount $show_low) </p>";
    }

    #If the order did succeed it will print the order
    if($order_id != null){
        echo "<p> Order was a success! Your order ID is $order_id </p>";
    }
}
```

Unset

```
drop procedure if exists checkout;
#Checkout procedure
drop procedure if exists checkout;
delimiter //
create procedure checkout(
    IN p_customer_id int,
    OUT p_order_id int,
    out p_out_of_stock_product int
)
begin
    declare temp_product_id int;
    declare temp_quantity int;
    declare temp_price decimal(6,2);
    declare temp_stock int;
    declare done int default false;

    declare cur1 cursor for select product_id, quantity from ItemInCart where customer_id = p_customer_id;
    declare continue handler for not found set done = True;

    start transaction;
    insert into OrderInformation(customer_id, date_ordered, order_status, total)
        values(p_customer_id, CURRENT_TIMESTAMP, 'processing', 0);

    set p_order_id = LAST_INSERT_ID();

    Open cur1;
        read_loop: LOOP
            fetch cur1 into temp_product_id, temp_quantity;

            if done then
                leave read_loop;
            end if;

            select stock_quantity into temp_stock from Product where product_id = temp_product_id;

            if temp_stock >= temp_quantity then
                select price into temp_price from Product where product_id = temp_product_id;

                insert into OrderItem(order_id, product_id, quantity, price)
                    values(p_order_id, temp_product_id, temp_quantity, temp_price);

                update Product
                    set stock_quantity = stock_quantity - temp_quantity
                    where product_id = temp_product_id;
                else
                    rollback;

                    set p_out_of_stock_product = temp_product_id;
                    set p_order_id = NULL;
                    close cur1;
                    leave read_loop;
                end if;
            end LOOP;

        close cur1;

        delete from ItemInCart where customer_id = p_customer_id;

        update OrderInformation

        set total =
        (
            select SUM(quantity * price)
            from OrderItem
            where order_id = p_order_id
        )
        where order_id = p_order_id;

        commit;

        set p_out_of_stock_product = NULL;
        end //

delimiter ;
```

3. (10 pts) Describe how your code prevents SQL injection.

- a. Explain where user input is accepted in your project (e.g., login, registration) and how these inputs are safely handled. Provide an example of technique (e.g., prepared statements or parameterized queries) used to protect against injection.

User input is accepted with logins, and registration, along with password changes. We used a similar method for all three, and that is with prepared statements. This can be seen with the login, the login page uses this statement

```
if (authenticate2($ POST["username"], $ POST["password"]) ==1)
```

Before allowing the user to login. The authenticate uses a prepared statement. What prepared statements do is it makes sure that the SQL statement is compiled separately from the data inserted, this makes sure that if a user enters SQL code in order to try to cause an injection the machine won't read the code as SQL code. It uses a bindParam to make sure the variable is seen as a string and not code. Since the inputs are never directly read as a SQL statement, SQL injections cannot happen.

```
$statement = $dbh->prepare("SELECT count(*) FROM Customer ".  
    "where username = :username and password =  
    sha2(:password,256) ");  
  
$statement->bindParam(":username", $user);  
$statement->bindParam(":password", $password);
```

The others that don't use authenticate like registerCustomer still use prepare statements and bindParam statements for all the variables which also make them secure.

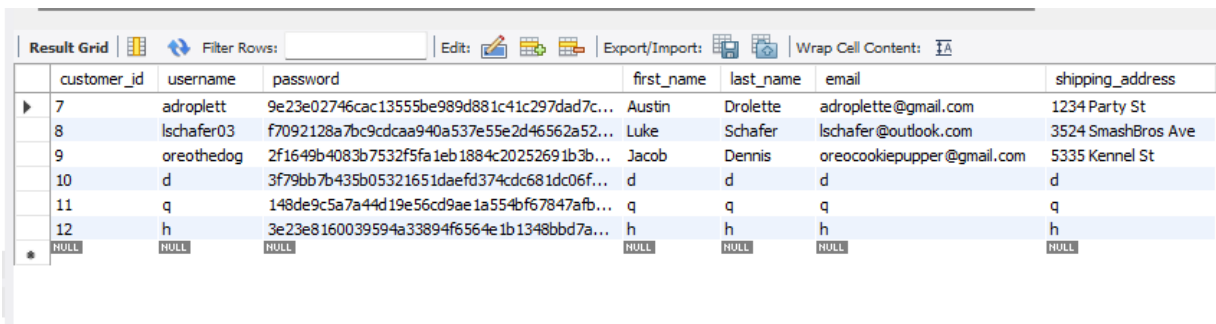
4. (10 pts) Describe how you encrypt user passwords.

Show the code where password encryption is applied (e.g., during registration or password change). Also explain what the stored password looks like in the database (e.g., a bcrypt hash like \$2b\$12\$. . .), and confirm that it is not stored as plain text.

We encrypt passwords using the sha2() function in mysql, we take the user's password in in a field like registration or password change, and then hash it and insert the hash into the database to be stored, we then compare the stored hash to the sha2() hash of the input password to validate logins.

```
function checkPasswordValidity( $user, $password ) {  
    //getting if the employee needs to reset their password as its their first use  
    try{  
        $dbh = connectDB();  
        $statement = $dbh->prepare("SELECT count(*) FROM Employee ".  
            "where username = :username and password = sha2(:password,256) and password_updated = false");  
        $statement->bindParam(":username", $user);  
        $statement->bindParam(":password", $password);  
        $result = $statement->execute();  
        $reset=$statement->fetch();  
        $dbh=null;  
  
        return $row[0];  
    } catch (PDOException $e) {  
        print "Error!" . $e->getMessage() . "<br/>";  
        die();  
    }  
}
```

checkPasswordValidity() is an example of how passwords are hashed before being compared or stored



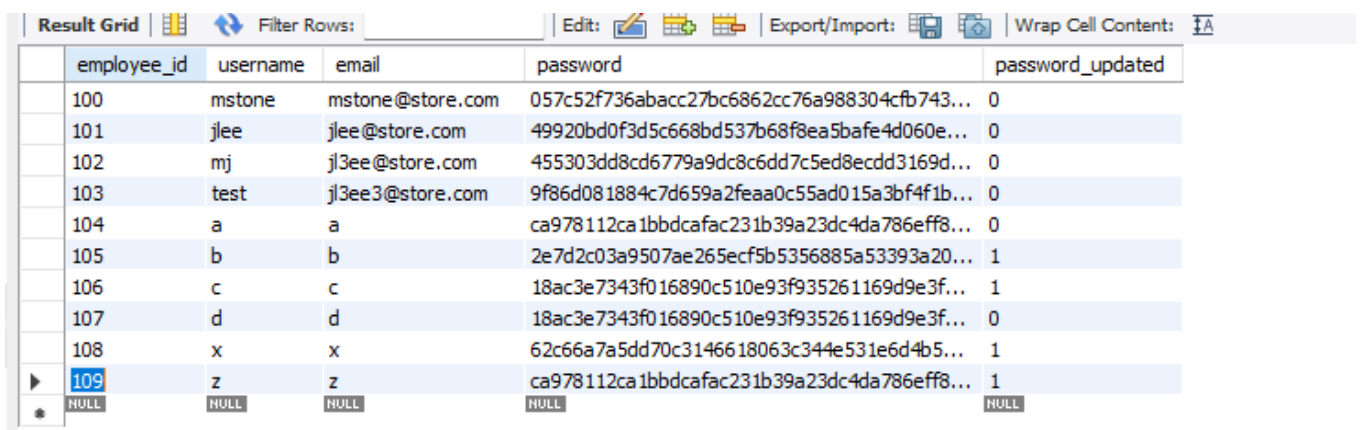
The screenshot shows a database table with columns: customer_id, username, password, first_name, last_name, email, and shipping_address. The password column contains long alphanumeric strings, which are SHA-256 hashes of the original passwords. The first_name, last_name, email, and shipping_address columns contain the corresponding user information.

customer_id	username	password	first_name	last_name	email	shipping_address
7	adroplett	9e23e02746cac13555be989d881c41c297dad7c...	Austin	Drolette	adroplett@gmail.com	1234 Party St
8	lschafer03	f7092128a7bc9cdcaa940a537e55e2d46562a52...	Luke	Schafer	lschafer@outlook.com	3524 SmashBros Ave
9	oreothedog	2f1649b4083b7532f5fa1eb1884c20252691b3b...	Jacob	Dennis	oreocookiepupper@gmail.com	5335 Kennel St
10	d	3f79bb7b435b05321651daefd374cdc681dc06f...	d	d	d	d
11	q	148de9c5a7a44d19e56cd9ae1a554bf67847afb...	q	q	q	q
12	h	3e23e8160039594a33894f6564e1b1348bbd7a...	h	h	h	h
*	NULL	NULL	NULL	NULL	NULL	NULL

In the screenshot above it can be seen that the password fields are filled with hashes.

5. (10 pts) Describe how employees are forced to change their password upon first login. Explain your logic and any database flags used to trigger the change.

When an employee account is made in the database, an attribute, password_update, is stored as false, password_update being false means that the employee's password has not been updated. Then when an employee logs in, we validate the login, then check if password_update is false, if it is, we redirect the employee to the passwordReset.php page, which upon successfully resetting their password, we also set the password_update field to true, so they will not need to reset their password when the log in again



employee_id	username	email	password	password_updated
100	mstone	mstone@store.com	057c52f736abacc27bc6862cc76a988304cfb743...	0
101	jlee	jlee@store.com	49920bd0f3d5c668bd537b68f8ea5baf4d060e...	0
102	mj	jl3ee@store.com	455303dd8cd6779a9dc8c6dd7c5ed8ecdd3169d...	0
103	test	jl3ee3@store.com	9f86d081884c7d659a2feaa0c55ad015a3bf4f1b...	0
104	a	a	ca978112ca1bbdcafac231b39a23dc4da786eff8...	0
105	b	b	2e7d2c03a9507ae265ecf5b5356885a53393a20...	1
106	c	c	18ac3e7343f016890c510e93f935261169d9e3f...	1
107	d	d	18ac3e7343f016890c510e93f935261169d9e3f...	0
108	x	x	62c66a7a5dd70c3146618063c344e531e6d4b5...	1
109	z	z	ca978112ca1bbdcafac231b39a23dc4da786eff8...	1
NULL	NULL	NULL	NULL	NULL

The password updated field can be seen as the rightmost column as false when an employee has not updated their password yet, and true when it has been updated

6. (10 pts) Explain how your system keeps track of product stock and price history.

We have a specific table that holds every update to a product, it stores a timestamp of the submitted change, the old price, new price, old stock, new stock, the updater (employee or customer), the updater's id, product id, and the action done (INSERT a new product, UPDATE an existing product, or DELETE a former product)

52	employee	106	NULL	2025-04-21 18:09:18	1	30.00	30.00	45	49	UPDATE
53	employee	106	NULL	2025-04-21 18:09:28	1	30.00	3.00	49	52	UPDATE
54	employee	106	NULL	2025-04-21 18:13:02	2	15.00	30.00	10	40	UPDATE

7. (5 pts) What was the best part you like about the project?

a. Explain why — was it a technical feature, the design, teamwork, or something else?

What we really enjoyed was how open ended it was. While we did have specific tasks to complete, they had many solutions, and just using the class notes was not quite enough to do everything. This gave the opportunity to search up new resources in order to figure out the problems we were having which are more effective for the learning process. We also enjoyed the teamwork aspect of the project, and how it was good practice with using gitHub and working from the same database. We also really enjoyed getting to work with HTML and PHP. I felt like it was the newest language I wanted to learn the most, and I was glad to get the chance to work on it.

8. (5 pts) What was the hardest part of this project, and how did you approach solving it? Include any specific tools, strategies, or resources you used.

The hardest part of the project was by far the viewOrderInformation section. Similar to the shoppingCart it was difficult being able to figure out how to get each result to print separately. We found a section on the PHP notes about using the `foreach($results as $row)` which helped with that section. But then what was difficult was trying to make viewing more information on the order possible when clicking it. We found this resource <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/details> which showed how using `<details>` it is possible to make more information available only by clicking on it. We also figured out how to make the tables more visible by using `<table>`. We had to do a lot of error testing, which we used AI for (shown below) but after a few hours we were able to get it working.

```
foreach ($results as $row) {  
  
    if ($currentOrder !== $row['order_id']) {  
  
        if ($currentOrder !== null) {  
  
            echo "</table></details><br>";  
  
        }  
  
        $currentOrder = $row['order_id'];  
  
        echo "<details>";  
  
        echo "<summary>Order ID: {$currentOrder} | Date:  
{ $row['date_ordered'] } | Total: \${ $row['total'] }</summary>";  
  
        echo "<table>";  
  
        echo "<tr><th>Product  
ID</th><th>Price</th><th>Quantity</th></tr>";  
  
    }  
}
```

```
        echo "<tr>";

        echo "<td>{$row['product_id']}</td>";

        echo "<td>\${$row['price']}</td>";

        echo "<td>{$row['quantity']}</td>";

        echo "</tr>";

    }

    if ($currentOrder !== null) {

        echo "</table></details>";

    }

    return $results;
```

9. (5 pts) Did you use any AI tools (e.g., ChatGPT, GitHub Copilot) during this project? If yes, describe what tools you used and what they helped with (e.g., code generation, debugging, planning).

We did use AI tools but not for a significant amount. We used ChatGTP for a tiny bit of planning. An example was trying to have the information be visible after being clicked with the orders information in the questions above. We asked how it is possible to implement something that allows more information to be visible after being clicked, and it recommended using <details> which we were then able to do searches for more information about later. We also used it for debugging, when we could not figure out what was causing a certain bug. We would give it our code with the error statement, and it did a very effective job of finding where the issue was and then showing us a solution.